



UNIVERSITÀ DI PARMA

Input & Output

*The future belongs to those who believe in the
beauty of their streams.*

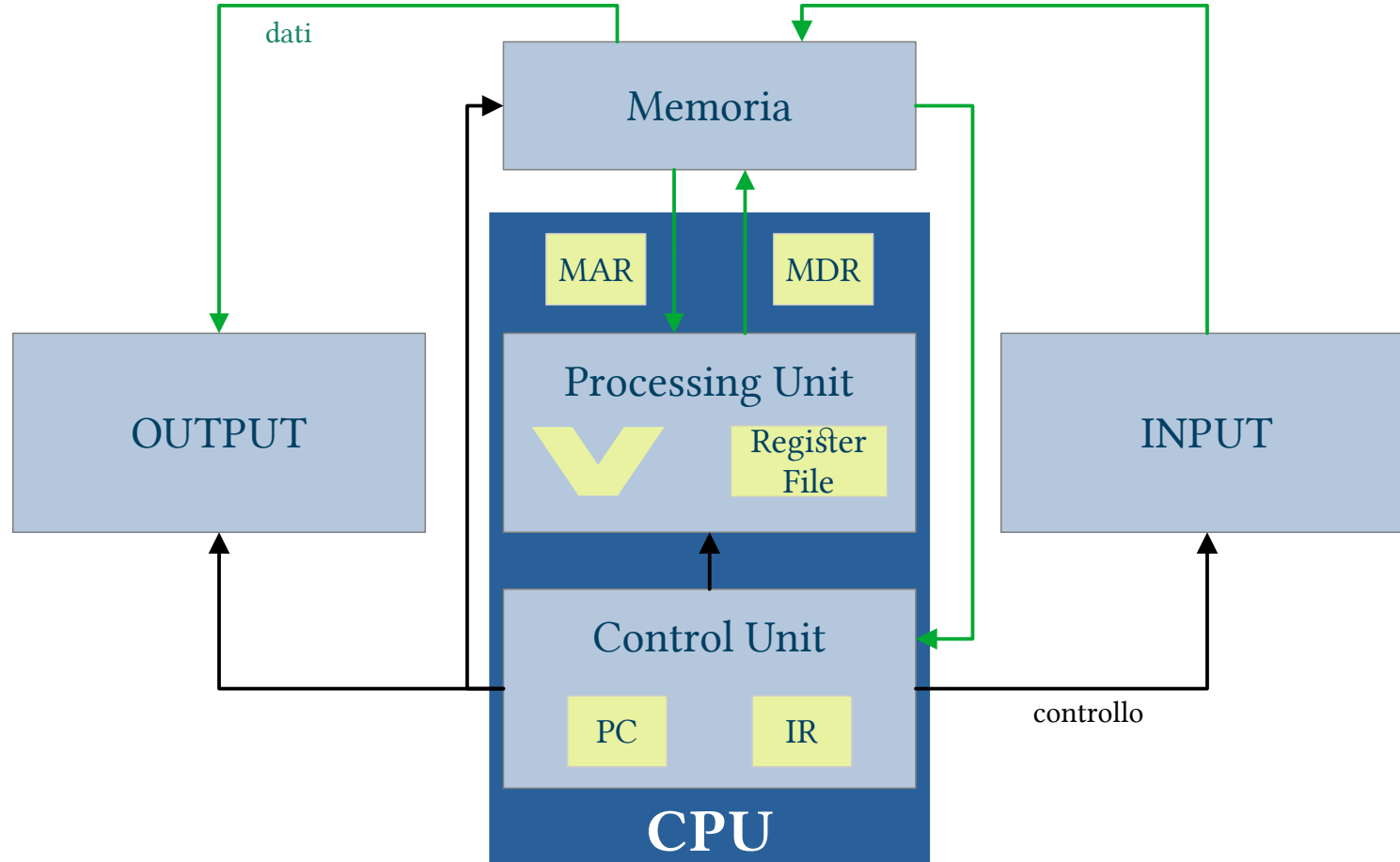
Eleanor Roosevelt (mod.)

- Non esaustivo → Manuale
- gli stream
- concetto di file
- principali operazioni
 - definizione
 - associare uno stream
 - lettura/scrittura e altre operazioni
 - File testuali e binari
 - chiusura
- I/O della console

SUMMARY



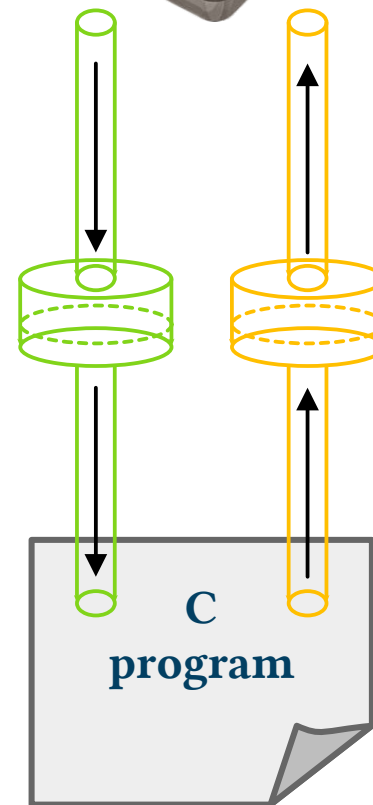
Von Neuman model



What is a stream?



- Gli stream sono una struttura ad alto livello che ci separa dall'HW
 - Canale di comunicazione
- Tipicamente I/O asincrono
 - Bufferizzato
- Input, output o entrambi



What is a stream?

- In C sono i “canali” di input o output
 - Già visto: tastiera e video
 - Ma ne possiamo avere altri
 - File, porte seriali, USB, ecc.
- Predefiniti
 - `stdin` → `scanf()`, ...
 - `stdout` → `printf()`, ...
 - `stderr`
- Vantaggio: un solo set di primitive per la gestione

- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- Si usa la struttura FILE
 - È un tipo di dato specifico
- Deve sempre essere un puntatore

```
FILE *miostream;  
FILE *altrostream, *altroancora;
```


- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- Dopo averlo definito possiamo “associare” uno stream
 - File
 - Periferica
 - ...
- Noi vedremo solo file

```
FILE *fopen(const char *pathname, const char *mode);
```

Cosa è un file?

- In italiano “archivio”
- Principale contenitore informazioni su supporti digitali
 - Hard disk
 - DVD/CD/BD
 - Memory stick
 - ...
- **Permanente**



Come si usa un file?

- Nastro magnetico
 - Testina che legge/scrive
 - Man mano che legge/scrive si sposta
 - Ma posso anche spostarmi in un punto specifico del nastro
- File
 - Concetto di “testina virtuale” ovvero posizione di lettura/scrittura
 - Ad ogni operazione “avanza” nel file
 - Di tante “posizioni” quanti sono i byte letti o scritti
 - Possibile saltare in posizioni specifiche



- Testuali o ASCII
 - Solo caratteri stampabili
 - Leggibili da essere umano
 - Esempi: codice sorgente C, file di configurazione (.ini),...
- File binari
 - Anche caratteri non stampabili
 - Esempi: file immagine, archivi compressi, videoscrittura,...
- Portabilità/Gestione $\leftrightarrow$ Spazio occupato



file.bin



file.txt

- Permette apertura file
- Due argomenti, entrambi stringhe:
 - Nome file (percorso relativo o assoluto), solo caratteri ASCII
 - Modalità apertura
- Restituisce
 - NULL se fallisce
 - Puntatore a “tipo” FILE se successo

```
FILE *fopen(const char *pathname, const char *mode);
```

r	lettura	Il file deve esistere
w	scrittura	Il file viene creato se non esistente o azzerato se esistente
a	scrittura a fine file	Il file viene creato se non esistente
r+	lettura e scrittura da inizio	Il file deve esistere
w+	azzeramento/creazione e lettura/scrittura	Il file viene creato se non esistente o azzerato se esistente
a+	lettura e scrittura a fine file	Il file viene creato se non esistente
b	In aggiunta ai precedenti se file binario	Esempi: “rb”, “w+b”, “wb+”...

- L'operazione di apertura può fallire:
 - File non esistente se richiesta lettura
 - Permessi
 - Errori di percorso...
- Fondamentale **controllare** quanto restituito da fopen()
- Utile l'uso di **void perror(const char *s)**
 - Mi indica perché operazione è fallita

- When opening a file we need a **file pointer** (FILE *), i.e.
 - `FILE *fp = fopen("mydata.dat", "rb");`
- The address inside the pointer is **not related** to the position of read or write operations within the file
- This pointer is used to handle all subsequent file operations
 - At that address the system stores all the data needed to deal with the file
 - Do not mess with it! Do not modify it!

- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- ASCII files, two main approaches
 - Raw I/O: read/write bytes (chars), strings...
 - Formatted I/O: thanks to a format string we can extract significant data
- Binary files
 - Raw I/O: read/write chunks of “memory” namely arrays or even single variables

Read/write primitives for ASCII files

- Single chars/bytes:
 - (f)getc() read a char from a stream
 - (f)putc() write a char to a stream
 - ungetc() put back a char in a stream
- Strings/Lines:
 - fgets() read an entire line as string from a stream up to a given size
 - fputs() write a string to a stream
- Formatted I/O:
 - fscanf() formatted read from a stream
 - fprintf() formatted write to a stream

- `int fgetc(FILE *stream);`
 - Reads a single char from a stream
 - Returns EOF on End Of File or error
 - This can help us to understand when the end of file is reached
- `int fputc(int c, FILE *stream);`
 - Write a single char on a file
 - Returns EOF on error
- `getc()` and `putc()` are aliases

`int ungetc(int c, FILE *stream);`

- Can be used to push back a char to a stream
- Namely, c is put as the first “byte” to be read next time
- Only one push back is guaranteed
- What is the reason why to put back a char?
 - Mixing `getc()/ungetc()` allows you to peek at the next character without consuming it.

- `char *fgets(char *s, int size, FILE *stream)`
 - Reads line from a stream up to size-1 chars or up to the first `'\n'`
 - Put it inside the array `s[]` adding the terminating null byte `'\0'`
 - Returns NULL on end of file or error
 - This can help us to understand when the end of file is reached
- `int fputs(const char *s, FILE *stream);`
 - Write the string `s` on a file without the `'\0'`
 - Returns EOF on error

- `int fprintf(FILE *stream, const char *format, ...);`
 - Like `printf()` is a formatted “print” on a file
 - Returned value can be ignored
- `int fscanf(FILE *stream, const char *format, ...);`
 - Can be used to read from text files
 - Format string can be exploited to “extract” relevant data
 - Returns the number of format specifier that have been “solved”
 - This can help us to understand when the end of file is reached

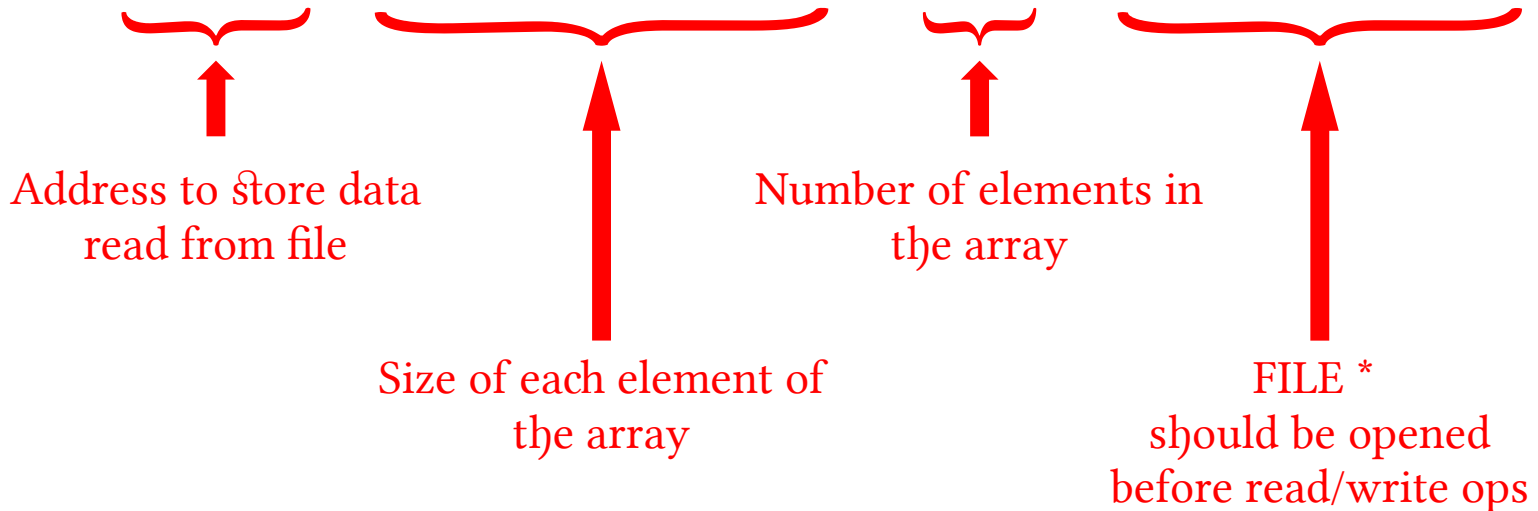
- I/O formattato vs non formattato
 - fgetc()/fputc() e fgets()/fputs() vs fprintf() e fscanf()
 - Preferire lettura formattata quando contenuto va interpretato
 - **Errore comune: leggo stringa e poi cerco di interpretarla in memoria**
- Le operazioni di I/O possono essere collo di bottiglia
 - Lettura/scrittura di singoli byte meno efficiente

- Scrivo/leggo un intero blocco di memoria
 - Indirizzo blocco
 - Dimensione singoli elementi
 - Numero elementi
 - Stream

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

- `fread()/fwrite()` can be used to read/write **arrays**
 - i.e. given an array like **`float data[127]`** we can read data from a file using a single statement:
 - `fread(data, sizeof(float), 127, filehandler);`



- `fread()/fwrite()` can be also used to read/write **single variables**
 - i.e. given a “simple” variable like **long len**, we can use it to store data read from a file using a single statement (like a *single element array*)
 - `fread(&len, sizeof(long), 1, filehandler);`

Address to store data
read from file
We need to use the ‘&’

Size of the “element”

We have only one
“element”

FILE *
should be opened
before read/write ops

- Some **read** operations **silently fails** when we try to read data beyond the end of a file
- Then, how to understand when we reached the end of file?
- 2 approaches:
 - Specific function `feof()`
 - Use the return value of functions used to read from file

- The `int feof(FILE *stream);` function returns true when a read operation failed because we reached the end of the file
 - Namely it works only when we **already tried to read** data!
 - Difficult to be directly used as a cycle condition
 - We usually need to:
 - Try to read data
 - Use `feof()` to check whether the read operations succeeded
 - If not, stop read operations, i.e., exit the read cycle using a `break`

- In order to check whether we are at the end of the file, it is **much more convenient** to use the return value of read functions:
 - fscanf() → number of format specifier that has been solved
 - fread() → number of bytes read from the file
 - fgets() → NULL when we can not read from the file
 - fgetc() → EOF constant when we can not read from the file
- Easy to directly use them as cycle conditions

Follow this approach with respect to the use of feof()!

- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- As previously said each read/write operation move the position for the next read/write operation, like a “virtual recording head”
- Often we need to move such position for different reasons, i.e.:
 - Skip part of a file, go to a specific position in a file (random access), restart from the beginning...
- Primitives:
 - `void rewind(FILE *stream);`
 - `int fseek(FILE *stream, long offset, int whence);`
 - `long ftell(FILE *stream);`

- **int fseek(FILE *stream, long offset, int whence);**
 - Move the “recording head” to a specific position
 - Offset is the relative position ($\pm$) wrt whence
 - Whence can assume the following values:
 - SEEK_CUR (current position),
 - SEEK_SET (beginning of the file)
 - SEEK_END (end of the file)
- **void rewind(FILE *stream);**
 - Move to the beginning of the file
 - Alias for `fseek(stream, 0, SEEK_SET)`
- **long ftell(FILE *stream);**
 - Returns the read/write position in a file

- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- Buona norma chiudere file quando non più necessario
 - Massimo numero file aperti contemporaneamente
 - Svuoto buffer
- Disassocia stream
- Puntatore riutilizzabile

```
int fclose(FILE *stream);
```

- Come accennato anche la console (tastiera + video) è associata a stream
 - `stdin` → stream di input (tastiera)
 - `stdout` → stream di output (video)
 - `stderr` → stream di output (messaggi di errore)
- Possibile uso delle funzioni di I/O
 - Ma esistono alias di molte
 - `fprintf(stdin, ...` → `printf(...`

- Caratteri

- `fgetc()` → `int getchar(void);`
- `fputc()` → `int putchar(int c);`

- Stringhe

- `fputs()` → `int puts(const char *s);`
- `fgets()` → ~~`char *gets(char *s);`~~ //DEPRECATED

- La libreria del C *bufferizza* l'accesso ai flussi di I/O
 - Ciò che scrivo da tastiera finisce in un'area di memoria che sarà man mano “consumata” dalle operazioni di lettura
 - Tipicamente il buffer viene riempito alla pressione del tasto di invio
 - Va posta attenzione alla gestione dell'input da console



UNIVERSITÀ DI PARMA

Input & Output



Don't cross the streams.

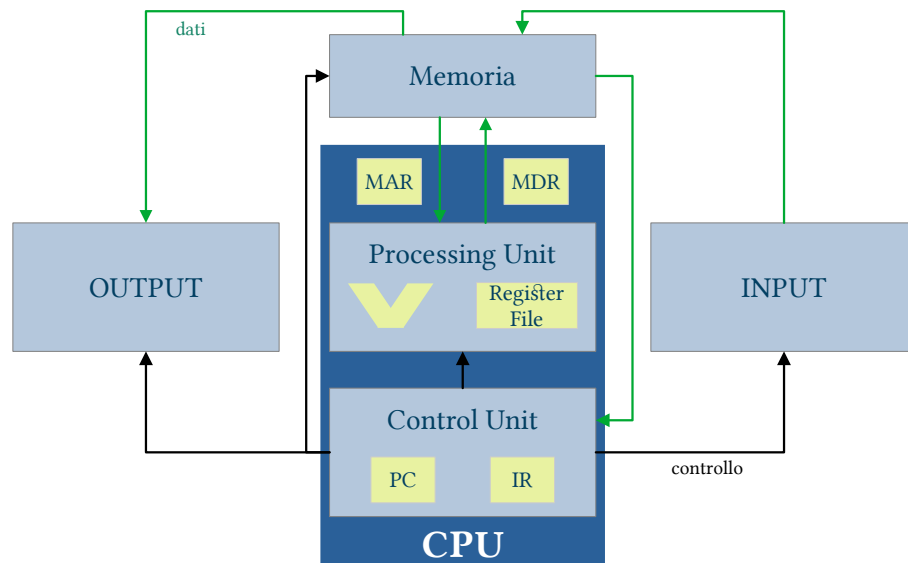
Egon, 1984



- Non esaustivo → Manuale
- gli stream
- concetto di file
- principali operazioni
 - definizione
 - associare uno stream
 - lettura/scrittura e altre operazioni
 - File testuali e binari
 - chiusura
- I/O della console

SUMMARY

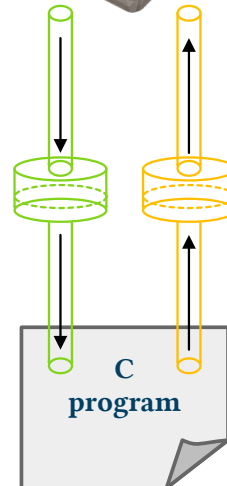




What is a stream?



- Gli stream sono una struttura ad alto livello che ci separa dall'HW
 - Canale di comunicazione
- Tipicamente I/O asincrono
 - Bufferizzato
- Input, output o entrambi





- In C sono i “canali” di input o output
 - Già visto: tastiera e video
 - Ma ne possiamo avere altri
 - File, porte seriali, USB, ecc.
- Predefiniti
 - stdin → scanf(), ...
 - stdout → printf(), ...
 - stderr
- Vantaggio: un solo set di primitive per la gestione



- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura



- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura



- Si usa la struttura FILE
 - È un tipo di dato specifico
- Deve sempre essere un puntatore

```
FILE *miostream;  
FILE *altrostream, *altroancora;
```




- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- Dopo averlo definito possiamo “associare” uno stream
 - File
 - Periferica
 - ...
- Noi vedremo solo file

```
FILE *fopen(const char *pathname, const char *mode);
```

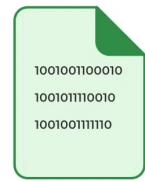
- In italiano “archivio”
- Principale contenitore informazioni su supporti digitali
 - Hard disk
 - DVD/CD/BD
 - Memory stick
 - ...
- **Permanente**



- Nastro magnetico
 - Testina che legge/scrive
 - Man mano che legge/scrive si sposta
 - Ma posso anche spostarmi in un punto specifico del nastro
- File
 - Concetto di “testina virtuale” ovvero posizione di lettura/scrittura
 - Ad ogni operazione “avanza” nel file
 - Di tante “posizioni” quanti sono i byte letti o scritti
 - Possibile saltare in posizioni specifiche



- Testuali o ASCII
 - Solo caratteri stampabili
 - Leggibili da essere umano
 - Esempi: codice sorgente C, file di configurazione (.ini),...
- File binari
 - Anche caratteri non stampabili
 - Esempi: file immagine, archivi compressi, videoscrittura,...
- Portabilità/Gestione ↔ Spazio occupato



file.bin



file.txt



- Permette apertura file
- Due argomenti, entrambi stringhe:
 - Nome file (percorso relativo o assoluto), solo caratteri ASCII
 - Modalità apertura
- Restituisce
 - NULL se fallisce
 - Puntatore a "tipo" FILE se successo

```
FILE *fopen(const char *pathname, const char *mode);
```



r	lettura	Il file deve esistere
w	scrittura	Il file viene creato se non esistente o azzerato se esistente
a	scrittura a fine file	Il file viene creato se non esistente
r+	lettura e scrittura da inizio	Il file deve esistere
w+	azzeramento/creazione e lettura/scrittura	Il file viene creato se non esistente o azzerato se esistente
a+	lettura e scrittura a fine file	Il file viene creato se non esistente
b	In aggiunta ai precedenti se file binario	Esempi: "rb", "w+b", "wb+"...



- L'operazione di apertura può fallire:
 - File non esistente se richiesta lettura
 - Permessi
 - Errori di percorso...
- Fondamentale **controllare** quanto restituito da fopen()
- Utile l'uso di **void perror(const char *s)**
 - Mi indica perché operazione è fallita



- When opening a file we need a **file pointer** (FILE *), i.e.
 - `FILE *fp = fopen("mydata.dat", "rb");`
- The address inside the pointer is **not related** to the position of read or write operations within the file
- This pointer is used to handle all subsequent file operations
 - At that address the system stores all the data needed to deal with the file
 - Do not mess with it! Do not modify it!



- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura

- ASCII files, two main approaches
 - Raw I/O: read/write bytes (chars), strings...
 - Formatted I/O: thanks to a format string we can extract significant data
- Binary files
 - Raw I/O: read/write chunks of “memory” namely arrays or even single variables



- Single chars/bytes:
 - (f)getc() read a char from a stream
 - (f)putc() write a char to a stream
 - ungetc() put back a char in a stream
- Strings/Lines:
 - fgets() read an entire line as string from a stream up to a given size
 - fputs() write a string to a stream
- Formatted I/O:
 - fscanf() formatted read from a stream
 - fprintf() formatted write to a stream



- `int fgetc(FILE *stream);`
 - Reads a single char from a stream
 - Returns EOF on End Of File or error
 - This can help us to understand when the end of file is reached
- `int fputc(int c, FILE *stream);`
 - Write a single char on a file
 - Returns EOF on error
- `getc()` and `putc()` are aliases



- Can be used to push back a char to a stream
- Namely, c is put as the first “byte” to be read next time
- Only one push back is guaranteed
- What is the reason why to put back a char?
 - Mixing `getc()/ungetc()` allows you to peek at the next character without consuming it.



- `char *fgets(char *s, int size, FILE *stream)`
 - Reads line from a stream up to size-1 chars or up to the first '\n'
 - Put it inside the array s[] adding the terminating null byte '\0'
 - Returns NULL on end of file or error
 - This can help us to understand when the end of file is reached
- `int fputs(const char *s, FILE *stream);`
 - Write the string s on a file without the '\0'
 - Returns EOF on error



- `int fprintf(FILE *stream, const char *format, ...);`
 - Like `printf()` is a formatted “print” on a file
 - Returned value can be ignored
- `int fscanf(FILE *stream, const char *format, ...);`
 - Can be used to read from text files
 - Format string can be exploited to “extract” relevant data
 - Returns the number of format specifier that have been “solved”
 - This can help us to understand when the end of file is reached



- I/O formattato vs non formattato
 - `fgetc()/fputc()` e `fgets()/fputs()` vs `fprintf()` e `fscanf()`
 - Preferire lettura formattata quando contenuto va interpretato
 - **Errore comune: leggo stringa e poi cerco di interpretarla in memoria**
- Le operazioni di I/O possono essere collo di bottiglia
 - Lettura/scrittura di singoli byte meno efficiente

- Scrivo/leggo un intero blocco di memoria
 - Indirizzo blocco
 - Dimensione singoli elementi
 - Numero elementi
 - Stream

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```



- `fread()/fwrite()` can be used to read/write **arrays**
 - i.e. given an array like **float data[127]** we can read data from a file using a single statement:
 - `fread(data, sizeof(float), 127, filehandler);`

Address to store data read from file

Size of each element of the array

Number of elements in the array

FILE *
should be opened before read/write ops



- `fread()/fwrite()` can be also used to read/write **single variables**
 - i.e. given a “simple” variable like **long len**, we can use it to store data read from a file using a single statement (like a *single element array*)
 - `fread(&len, sizeof(long), 1, filehandler);`

Address to store data
read from file
We need to use the '&'
Size of the “element”
We have only one
“element”
FILE *
should be opened
before read/write ops



- Some **read** operations **silently fails** when we try to read data beyond the end of a file
- Then, how to understand when we reached the end of file?
- 2 approaches:
 - Specific function `feof()`
 - Use the return value of functions used to read from file



- The `int feof(FILE *stream);` function returns true when a read operation failed because we reached the end of the file
 - Namely it works only when we **already tried to read** data!
 - Difficult to be directly used as a cycle condition
 - We usually need to:
 - Try to read data
 - Use `feof()` to check whether the read operations succeeded
 - If not, stop read operations, i.e., exit the read cycle using a break



- In order to check whether we are at the end of the file, it is **much more convenient** to use the return value of read functions:
 - fscanf() → number of format specifier that has been solved
 - fread() → number of bytes read from the file
 - fgets() → NULL when we can not read from the file
 - fgetc() → EOF constant when we can not read from the file
- Easy to directly use them as cycle conditions

Follow this approach with respect to the use of feof()!



- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura



- As previously said each read/write operation move the position for the next read/write operation, like a “virtual recording head”
- Often we need to move such position for different reasons, i.e.:
 - Skip part of a file, go to a specific position in a file (random access), restart from the beginning...
- Primitives:
 - `void rewind(FILE *stream);`
 - `int fseek(FILE *stream, long offset, int whence);`
 - `long ftell(FILE *stream);`

- **int fseek(FILE *stream, long offset, int whence);**
 - Move the “recording head” to a specific position
 - Offset is the relative position ($\pm$) wrt whence
 - Whence can assume the following values:
 - SEEK_CUR (current position),
 - SEEK_SET (beginning of the file)
 - SEEK_END (end of the file)
- **void rewind(FILE *stream);**
 - Move to the beginning of the file
 - Alias for `fseek(stream, 0, SEEK_SET)`
- **long ftell(FILE *stream);**
 - Returns the read/write position in a file



- Definire uno stream
- Associare uno stream (ovvero “aprire”)
- Lettura o scrittura dati
 - Altre operazioni
- Chiusura



- Buona norma chiudere file quando non più necessario
 - Massimo numero file aperti contemporaneamente
 - Svuoto buffer
- Disassocia stream
- Puntatore riutilizzabile

```
int fclose(FILE *stream);
```



- Come accennato anche la console (tastiera + video) è associata a stream
 - `stdin` → stream di input (tastiera)
 - `stdout` → stream di output (video)
 - `stderr` → stream di output (messaggi di errore)
- Possibile uso delle funzioni di I/O
 - Ma esistono alias di molte
 - `fprintf(stdin, ...` → `printf(...`



- Caratteri

- fgetc() → `int getchar(void);`
- fputc() → `int putchar(int c);`

- Stringhe

- fputs() → `int puts(const char *s);`
- fgets() → ~~`char *gets(char *s);`~~ //DEPRECATED



- La libreria del C *bufferizza* l'accesso ai flussi di I/O
 - Ciò che scrivo da tastiera finisce in un'area di memoria che sarà man mano "consumata" dalle operazioni di lettura
 - Tipicamente il buffer viene riempito alla pressione del tasto di invio
 - Va posta attenzione alla gestione dell'input da console

